Aarogya Drishti

AI-assisted oral cancer screening, telemedicine and patient care platform for remote tribal communities.

Built for the AAROHAN Inter-Centre Hackathon 2026 (organised by ITDA Paderu, Alluri Sitharama Raju District, Andhra Pradesh).

> NOT FOR CLINICAL USE. This is a hackathon demonstration. It uses synthetic patients and licensed demo images only. All AI output is advisory, research-only and never a diagnosis.

Overview

One working platform that carries a case from village-level oral screening to specialist diagnosis, treatment linkage and district reporting.

Core workflow

1. Register patient and record consent
2. Capture oral photos on a smartphone
3. Image quality check
4. Experimental AI analysis (advisory only)
5. Medical Officer review and signed decision
6. Video consultation or physical referral
7. Biopsy / histopathology if needed
8. Treatment tracking and navigation
9. District dashboard

Users and roles

Role	Main responsibilities
Health Worker	Register patients, consent, screening intake, photos
Medical Officer (MO)	Review cases, sign decisions, create referrals, book visits
Specialist	Consult (video/physical), order tests, enter final diagnosis
Navigator	Follow-up tasks, contact log, missed-visit rescheduling
District Admin	Aggregate dashboard and export (no patient identifiers)

Non-negotiable rules

1. AI output is advisory and stored separately from the signed clinician conclusion.
2. The app never confirms or stages cancer by itself and never tells a patient they are cancer-free. Only a clinician-entered pathology result can mark a case as confirmed in the simulation.
3. Every mock or research model output carries the label `NOT FOR CLINICAL USE`, model version and timestamp.
4. The clinical workflow must keep working if the AI service is down.
5. Every API call is authenticated and authorized at patient/facility level, not just by role (deny by default).
6. No bounding boxes or heatmaps unless a real supervised localization model exists.
7. No secrets in Git. Only `.env.example` with placeholders.
8. Offline records are marked synced only after server acknowledgement.

Tech stack

Layer	Choice
Frontend	React + TypeScript + Vite PWA (react-i18next, Telugu/English)
Backend	FastAPI, SQLAlchemy, Alembic, Pydantic
Database	PostgreSQL
Image storage	Private S3-compatible bucket (MinIO locally), short-lived signed URLs
Image QC	OpenCV (blur, brightness, glare)
AI	Pluggable adapter: `mock-v1` default, optional EfficientNet-B0 research model
Video	Approved SDK (Jitsi JaaS, Daily, LiveKit) with time-limited tokens
Offline	Service worker + IndexedDB queue with idempotency keys
Deploy	Docker Compose locally; managed container/VM + managed Postgres online
Monitoring	Structured logs (no PHI), `/healthz`, uptime check

Repository structure

```
aarogya-drishti/
  apps/web/        # PWA: health worker, clinician, navigator, admin UI
  services/api/    # FastAPI, auth, workflow, migrations
  services/ai/     # quality checker + mock/research adapters
  infra/           # docker-compose, deployment manifests
  docs/            # PRD, architecture, ER, API, threat model, test report
```

```
  tests/           # unit, integration, E2E, security
  .env.example
  README.md
```

Quick start

```
git init aarogya-drishti && cd aarogya-drishti
mkdir -p apps/web services/api services/ai docs tests infra

# Frontend
cd apps && npm create vite@latest web -- --template react-ts
cd web && npm install && cd ../..

# Backend
cd services/api && python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install fastapi 'uvicorn[standard]' sqlalchemy alembic \
  'psycopg[binary]' pydantic-settings python-multipart
pip freeze > requirements.txt
```

Run the full stack

```
cp .env.example .env               # fill in placeholders; never commit .env
docker compose up --build -d       # web, api, ai, postgres, minio
alembic upgrade head               # run migrations (from services/api)
make seed                          # load synthetic demo data
curl http://localhost:8000/healthz # expect 200
```

Demo data can be reset any time with `make reset`.

Build order (vertical slices, tests pass before moving on)

1. Docker stack + health check + migrations + seed data
2. Auth, roles, facility scoping, audit middleware
3. Patient, consent, screening APIs and Telugu/English forms
4. Image upload, quality checks, mock AI, stored results
5. Review queue, signed review, referral
6. Appointments, video tokens, physical slip
7. Investigations, diagnosis, navigation tasks
8. Timeline and dashboard aggregates
9. Offline sync, security tests, deployment, documentation

Case state machine

`DRAFT > CONSENTED > SCREENED > AWAITING_CLINICIAN > REVIEWED > REFERRAL_PLANNED > APPOINTED > ASSESSED > DIAGNOSTICS_PENDING > RESULT_REVIEWED > CARE_LINKED | CLOSED`

Tracked exceptions: `RESCHEDULED`, `MISSED`, `DECLINED`, `LOST_CONTACT`.

No AI result may move a case forward by itself. Any invalid transition returns HTTP 409.

Key API routes

Route	Purpose
`POST /auth/login`, `POST /auth/mfa/verify`, `GET /me`	Authentication
`POST /patients`, `GET /patients/{id}`	Registration and longitudinal record
`POST /patients/{id}/consents`	Purpose-specific consent
`POST /screenings`, `POST /screenings/{id}/images`	Screening intake and photo upload
`POST /ai/analyze`, `GET /ai/analyses`	Non-clinical AI analysis
`GET /queue`, `POST /reviews`, `POST /referrals`	Clinical workflow
`GET /slots`, `POST /appointments`, `PATCH /appointments/{id}`	Booking
`POST /appointments/{id}/videotoken`	Time-limited video token
`POST /investigations`, `POST /diagnoses`	Tests and clinician diagnosis
`PATCH /navigation/{id}`	Navigator follow-up
`GET /dashboard/summary`, `GET /dashboard/export`	Aggregates only
`GET /healthz`	Health check

All routes require authentication, patient/facility authorization, validation, pagination on lists, and an `Idempotency-Key` header on retryable writes.

Image quality pipeline

- Validate extension and decode with Pillow/OpenCV (reject renamed non-images)
- Enforce max size (e.g. 8 MB) and minimum dimensions
- Fix EXIF orientation, strip metadata, compute SHA-256
- Blur (variance of Laplacian), brightness and glare checks trigger recapture

- Calibrate thresholds on demo images and record them in `docs/`

AI adapter levels: Level 1 mock adapter (required) · Level 2 EfficientNet-B0 research model (optional, only if the core system is complete; requires `DATASET_CARD.md` and `MODEL_CARD.md`) · Level 3 real clinical use (never allowed).

Telugu and offline-first

- `te.json` / `en.json` dictionaries, no hardcoded English, language toggle on every page
- Noto Sans Telugu bundled locally (UI and PDF)
- 44px+ touch targets, icon + label buttons
- IndexedDB drafts; each write has a client UUID + Idempotency-Key
- Sync states: pending, uploading, confirmed, failed
- Jury test: airplane mode, create patient, reopen, reconnect, sync once, exactly one server record

Security checklist

- Argon2/bcrypt, short session expiry, MFA for MO/specialist/admin, no shared accounts
- Role AND patient/facility check in every query; deny by default
- HTTPS, encrypted DB and backups, private bucket, signed URLs under 5 minutes
- Safe uploads: decode, size limit, safe filenames, no path traversal
- Append-only audit log (no PHI); `audit_events`, `consents`, `case_state_history`, `appointment_status_history` are append-only
- Secrets only via env vars or a secret manager
- Threats covered: stolen phone, replayed token, IDOR, mass assignment, leaked signed URL, malicious upload, shared video link, backup leak

Testing

Pytest unit tests, API integration tests, Playwright mobile E2E, authorization tests, offline demo recording and a known-issues list. Write failing security tests first, then fix.

Mandatory scenarios include: Telugu switch, offline sync, blurry image, unauthorized access (403), doctor overriding AI, missed appointment follow-up, network-loss recovery, audit trail, invalid transition (409), oversized image (413) and dashboard exact counts (12 cases, 4 referrals seeded).

Deployment

1. `docker compose up --build -d`
2. Run migrations, then `make seed`
3. Verify `GET /healthz` returns 200
4. Open the HTTPS URL on Android; test camera, reload and offline

5. Restart the API and confirm data persists
6. Confirm bucket is private and signed URLs expire
7. Run automated tests against the deployed URL
8. Back up the DB and restore into a fresh environment
9. After the event: disable test users and rotate secrets

Free hosting tiers may sleep or wipe files, so use persistent database/object storage and keep the local Docker route as a backup.

Nine-day plan

Day	Date	Goal
1	27 Oct	Plan: repo, PRD, schema draft, wireframes
2	28 Oct	Foundation: FastAPI skeleton, login, RBAC, PWA shell, Docker
3	29 Oct	Registration, consent, screening forms (Telugu)
4	30 Oct	Images, private storage, OpenCV QC, mock AI
5	31 Oct	Review, referral, state machine, offline drafts (end-to-end by today)
6	1 Nov	Video and booking
7	2 Nov	Diagnostics, tasks, dashboard
8	3 Nov	Harden, deploy, real-phone testing
9	4 Nov	Demo. Code freeze 10:00 AM IST

Judging criteria (100 marks)

Criterion	Marks
End-to-end workflow and integration	20
AI engineering, image quality, safe integration	15
MO review, video, appointments	15
Records, diagnostics, navigation	15
Security, privacy, access	15
Telugu and offline reliability	10

Criterion	Marks
Deployment, testing, documentation	10

Tie-breakers: patient workflow, then security/privacy, then AI safety/integration.

Submission checklist

- [ ] Git repo with daily commits and contribution record
- [ ] Deployed HTTPS demo URL + evaluator test accounts
- [ ] README (install, architecture, stack, credentials, known limits)
- [ ] Architecture diagram with trust boundaries; ER diagram
- [ ] Role-access matrix
- [ ] OpenAPI docs with examples
- [ ] Environment variable table (no secrets)
- [ ] AI model card, dataset card, validation and limitations
- [ ] Threat model
- [ ] Test report with failure cases and offline tests
- [ ] Deployment, backup and rollback guide
- [ ] Synthetic dataset + reset command
- [ ] Telugu UI demonstration
- [ ] 5-minute demo video
- [ ] Disclosure of AI tools, templates, third-party code
- [ ] Roadmap from demo to approved clinical pilot

Team rules

- Max 5 students per team; each student in one team only
- Open-book and AI-assisted coding allowed, with disclosure
- Disqualifiers: real patient data, presenting AI as clinical, exposed records, plagiarism, hardcoded secrets

Prize

Rs 10,000 per winning team.

Disclaimer

Aarogya Drishti is a research and training prototype. It is not a medical device, has no clinical validation, and must not be used to diagnose, screen or treat real patients. Any move toward real use requires an approved clinical pilot with DPDP Act and telemedicine compliance.